# Implementing a Transformer: Attention and Encoder Blocks (Lab 7)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, November 11, 2026

Instructor: Prof. Jongmin Lee

**How to use this note:**

- Read it before the lab session. The lab will not repeat this material.
- Covered: queries, keys and values, the scaling factor, masking, multiple heads, positional encoding, and the residual and normalization order inside an encoder block. The emphasis is on what each method assumes and where it breaks.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab07.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy (grading and AI use) was given in the policy slides of the first class and is not repeated here.

## 1 Learning objectives

1. Break one attention operation into four steps and state the tensor shape at each step. (Task 1)
2. Explain why the scores are divided by $\sqrt{d_k}$, and why a masked position must receive exactly zero weight. (Task 1)
3. Split a vector into heads and merge it back, getting both the shape and the contents right. (Task 2)
4. Show that self-attention is permutation equivariant, and invariant once the positions are averaged, and explain how positional encoding breaks that. (Task 3)
5. Fix the residual and layer-norm order to the spec, and design an ablation that changes one thing at a time. (Task 3, Task 4)

## 2 Background: which one went off first?

A machine tool has stopped. The maintenance engineer opens the log. Both a heat alarm and a pressure alarm are in there. One thing decides what happens next: which of them fired first. Heat first and the engineer takes the cooling circuit apart; pressure first and it is the hydraulics. The same alarms in the same numbers send the engineer to a different part of the machine depending on the order.

That is Lab 7. You get a log of events in time order and predict which of the two alarms came first.

Up to Lab 6 this kind of data went through a recurrent network, which reads the events one at a time and updates a memory as it goes. It works like a line of people passing a whisper along. The longer the line, the more the beginning of the story fades. Attention removes the line and lets everyone look at everyone directly.

There is a twist. *Self-attention* on its own cannot see order. Shuffle the events and the input looks the same to it. The one thing the engineer needs is structurally invisible.

So each slot gets a vector that says "you are the $n$th one", added to the input. In this lab you take that marker out and put it back, and see for yourself how much it is worth.

**The four steps of Lab 7:**

- **One attention:** scores, the scaling factor, the mask, softmax, the weighted sum. (Task 1)
- **Heads:** split the vector into heads, attend within each, merge them back. (Task 2)
- **Positions and the block:** build the positional encoding and wire the block's residuals and norms in the order the spec fixes. (Task 3)
- **Controlled ablation:** remove only the positional encoding and hold everything else fixed. (Task 4)

# 3 Key concepts

## 3.1 Self-attention: queries, keys, values

From the vector $x_j$ of one event you build three vectors that come from the same input and do different jobs (Figure 1).

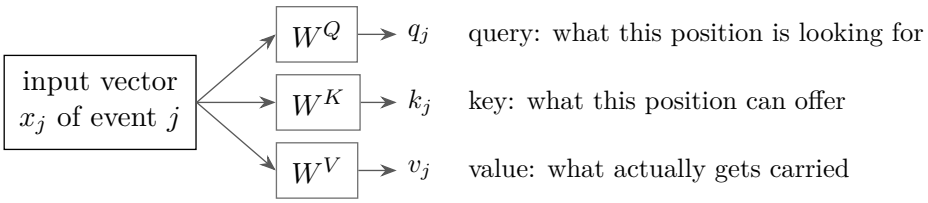



**Figure 1. Three vectors from one input.** Keeping the three roles separate is what lets "what am I looking for" and "what do I pass on" be learned independently.

**One attention operation:**

The output at position $i$ is a weighted average of all the values, and the weights come from the dot products of queries with keys.

$$\alpha_{ij} = \frac{\exp\big(q_i \cdot k_j/\sqrt{d_k}\big)}{\sum_{j'} \exp\big(q_i \cdot k_{j'}/\sqrt{d_k}\big)}, \qquad z_i = \sum_j \alpha_{ij}\, v_j.$$

- $d_k$ is the query and key dimension; one head in Lab 7 uses $d_k = 8$. $\alpha_{ij}$ is how much position $i$ looks at position $j$, and each row sums to 1.
- In matrix form it is one line, $\text{softmax}(QK^\top/\sqrt{d_k})V$. That line is Task 1.

**Assumptions and limits:**

- Nothing is assumed about the distance between positions. A far position is consulted at the same cost as the neighbouring one, and the price is that order information disappears entirely (§3.5 puts it back). Cost grows with the square of the length, which does not matter at length 16.

## 3.2 Why divide by $\sqrt{d_k}$?

Dot products grow with dimension. If the components of the query and the key are independent with variance 1, then $q \cdot k$ has variance $d_k$ and standard deviation $\sqrt{d_k}$.

- At $d_k = 64$ the scores have standard deviation 8 (draw the components at random 200,000 times and you measure about 8.00). Take two scores one standard deviation apart: unscaled, the scores $[8, 0]$ give softmax $[0.9997, 0.0003]$, while dividing by $\sqrt{64} = 8$ turns them into $[1, 0]$ with softmax $[0.7311, 0.2689]$.
- The gradient of softmax is roughly proportional to $p(1 - p)$. Those two cases give 0.000335 and 0.1966, a factor of about 590. Without the division, training effectively stops.

**Spec (Task 1)**

Divide `scores = Q @ K^T` by `sqrt(d_k)`, where `d_k = Q.shape[-1]`. Dividing by $d_k$, or by $\sqrt{d_{\text{model}}}$, is caught by the hidden tests.

**Assumptions and limits:**

- The number comes from assuming independent, unit-variance components. Training makes that assumption inexact, but the effect of matching the magnitude survives. Fixing the scale does not prevent saturation as such, so attention becoming peaked late in training is normal.

### 3.3 Masking: positions that must not be seen

To build a batch, logs of different lengths have to be padded to the same length; Lab 7 fills the tail of a short log with token 0. Those slots hold nothing, so attention must not see them (Figure 2).

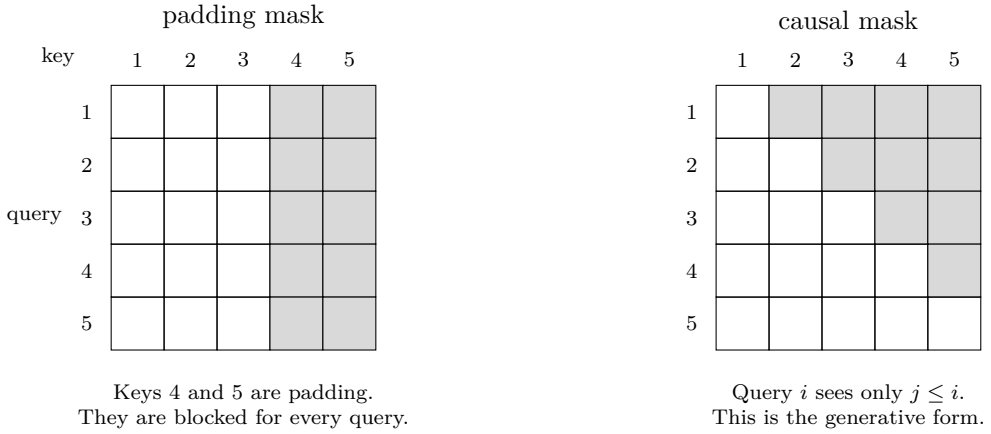



**Figure 2. Two masks.** Rows are queries and columns are keys. Shaded cells are blocked, and after softmax their weight has to be exactly zero. The Lab 7 experiment uses only the left one.

**How to block:**

- **Before** taking the softmax, set the scores of the blocked cells to $-\infty$. Then $\exp(-\infty) = 0$, the weight is exactly zero, and the remaining weights still sum to 1. Multiplying by zero **after** the softmax leaves the remaining weights summing to less than 1.

**Spec (Task 1)**

Keep the cells where the mask is `True` and set the scores where it is `False` to

`float("-inf")`. A mask of `None` blocks nothing.

**Assumptions and limits:**

- A fully blocked row makes the softmax 0/0, which returns NaN. Lab 7 blocks keys only, so that never happens. The **query** rows at padded positions are still computed, and their outputs are discarded by the pooling step that follows.

### 3.4 Why several heads

One attention operation produces a single weight distribution per query, which is not enough when a position has to look for two different things at once. Multi-head attention cuts the vector into pieces and lets each piece attend on its own (Figure 3).

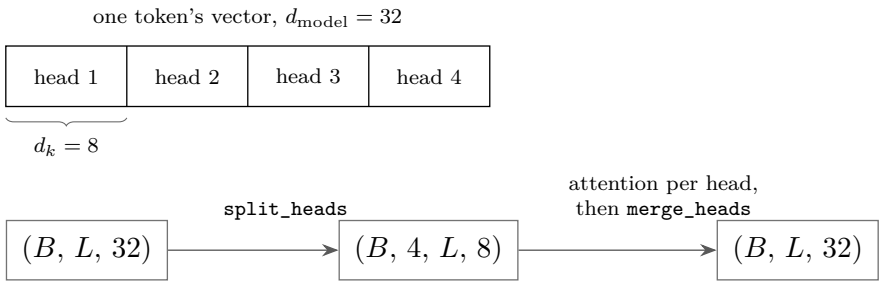



**Figure 3. Splitting into heads and merging back.** The axis being cut is the feature dimension, not the length. Each token's 32-dimensional vector becomes four 8-dimensional pieces, each attends on its own, and the pieces are concatenated back to 32.

**The reshape is where it goes wrong:**

- The correct order splits the last axis into $(4, 8)$ first, then moves the head axis in front of the length axis; merging reverses the move and then concatenates. The $W^O$ applied after merging is the only place where heads exchange information.
- Reshaping straight to $(B, 4, L, 8)$ gives the right shape with the wrong contents: it cuts **the sequence into four blocks** instead of the feature dimension. No error is raised, training runs, and only the accuracy looks strange.

**Assumptions and limits:**

- $d_{\text{model}}$ must be divisible by the number of heads. Adding heads leaves the total cost about the same, but shrinks $d_k$ per head and with it what one head can represent. There is also no guarantee that a head takes on a human-readable role.

### 3.5 Positional encoding: breaking permutation invariance

Self-attention does not know about position. Reorder the input and the output follows the same reordering with unchanged values (*permutation equivariance*), and the moment you average over positions the result does not change at all (*permutation invariance*): the order information is gone.

- This is a property of the function. For any weights at all, a permuted input and the original produce the same number, so more training changes nothing.

- So one vector per position is built and **added** to the input embeddings (added rather than concatenated, to keep the dimension the same). Lab 7 uses a fixed, unlearned table of sines and cosines whose period varies with position and dimension.

**Table 1. Positional encoding at $d_{\text{model}} = 4$.** Even columns are sines, odd columns cosines. The first two columns have a short period and change a lot between positions; the last two have a period 100 times longer and move slowly.

| position | col 0 (sin) | col 1 (cos) | col 2 (sin) | col 3 (cos) |
|---|---|---|---|---|
| 0 | 0.0000 | 1.0000 | 0.0000 | 1.0000 |
| 1 | 0.8415 | 0.5403 | 0.0100 | 1.0000 |
| 2 | 0.9093 | −0.4161 | 0.0200 | 0.9998 |
| 3 | 0.1411 | −0.9900 | 0.0300 | 0.9996 |

**Reading Table 1:**

- The angle for position $p$ and dimension pair $i$ is $p/10000^{2i/d_{\text{model}}}$, so a larger $i$ moves the angle more slowly. Row 0 is $0, 1, 0, 1$ because the angle there is 0.
- The cosine similarity between the row for position 0 and the others is 0.770 at position 1, 0.292 at position 2, and 0.005 at position 3. Nearby positions resemble each other and distant ones do not.

**Spec (Task 3)**
$\text{PE}[p,\, 2i] = \sin\big(p/10000^{2i/d_{\text{model}}}\big)$, $\text{PE}[p,\, 2i+1] = \cos\big(p/10000^{2i/d_{\text{model}}}\big)$. The returned shape is $(1, \texttt{max_len}, \texttt{d_model})$ in float32, and $p$ counts from 0.

**Assumptions and limits:**

- Being a fixed table, it applies to lengths never seen in training, but it cannot learn a position representation suited to this task. Many models use learned position embeddings instead.
- It encodes absolute position, so tasks where relative distance matters ("three slots later") are better served by a relative scheme.

### 3.6 The encoder block: residual and norm order

An encoder block wraps attention and a position-wise FFN, each in a residual connection followed by layer normalization (Figure 4). Lab 7 fixes the *post-norm* order of the original paper.

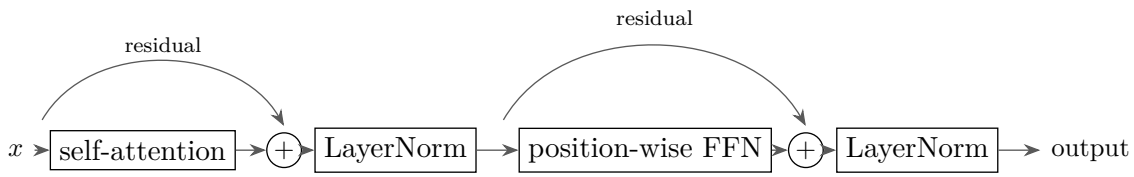



**Figure 4. The data path of an encoder block.** Each residual adds a sub-layer's own input to that sub-layer's output: $x$ for the attention, the first norm's output $h$ for the FFN. Normalizing after adding is what post-norm means.

**Spec (Task 3)**
$h = \text{LayerNorm}_1\big(x + \text{Attention}(x)\big)$, output $= \text{LayerNorm}_2\big(h + \text{FFN}(h)\big)$. The two residuals add the block input $x$ and the output $h$ of the first norm.

**What each piece does:**

- **Residual connection:** the input passes through unchanged even when the sub-layer does nothing. This is why gradients survive as blocks are stacked.
- **Layer normalization:** each position's vector is centred and scaled within that position. Normalizing over the feature axis rather than the batch makes it independent of batch size.
- **Position-wise FFN:** the same two-layer network is applied at every position. Attention is the only place where positions mix.

**Assumptions and limits:**

- Pre-norm, $x + \text{Attention}(\text{LayerNorm}(x))$, which normalizes *before* the sub-layer, is also widely used and is more stable for deep models, but it is a different architecture and does not pass the Lab 7 tests. Dropping a residual altogether also raises no error, since the shapes match.
- Real models stack 6 to several dozen of these blocks. Lab 7 uses one, which limits what it can represent.

## 4 Worked example: three events, $d_k = 2$

Work one attention operation through to the end with the queries, keys, and values in Table 2. With three events the attention matrix is $3 \times 3$, and Table 3 holds the first three steps.

**Table 2. The input of the worked example.** Queries, keys, and values for three events. With only two dimensions the dot products can be done in your head, and the integer values make the weighted sum easy to check.

| **event** | **query** $q_i$ | **key** $k_i$ | **value** $v_i$ |
|---|---|---|---|
| 1 | (1, 0) | (2, 0) | (1, 0) |
| 2 | (0, 1) | (0, 2) | (0, 1) |
| 3 | (1, 1) | (1, 1) | (2, 2) |

**Table 3. Steps 1 to 3.** The raw scores are integers, and row 1's exponentials sum to $4.1133 + 1 + 2.0281 = 7.1414$, so $4.1133/7.1414 = 0.5760$. Row 2 mirrors row 1, and row 3 is uniform because its three scores are equal.

| | **Step 1: $QK^\top$** | | | **Step 2: $\div\sqrt{2}$** | | | **Step 3: softmax** | | |
|---|---|---|---|---|---|---|---|---|---|
| **query** | key1 | key2 | key3 | key1 | key2 | key3 | key1 | key2 | key3 |
| 1 | 2 | 0 | 1 | 1.4142 | 0 | 0.7071 | 0.5760 | 0.1400 | 0.2840 |
| 2 | 0 | 2 | 1 | 0 | 1.4142 | 0.7071 | 0.1400 | 0.5760 | 0.2840 |
| 3 | 2 | 2 | 2 | 1.4142 | 1.4142 | 1.4142 | 0.3333 | 0.3333 | 0.3333 |

**Step 4: the weighted sum of the values:**

1. $z_1 = 0.5760\,(1,0) + 0.1400\,(0,1) + 0.2840\,(2,2) = (1.1440,\ 0.7080)$
2. $z_2 = (0.7080,\ 1.1440), \quad z_3 = \frac{1}{3}\big[(1,0) + (0,1) + (2,2)\big] = (1.0000,\ 1.0000)$

**The same input with a causal mask:**

- Row 1 keeps only key 1, so the weights are $(1,\ 0,\ 0)$ and the output is $(1.0000,\ 0.0000)$, different from the unmasked $(1.1440,\ 0.7080)$. The weight on a blocked cell is exactly 0, not 0 after rounding.
- In row 2 the scores $(0,\ 1.4142)$ give softmax and output both equal to $(0.1956,\ 0.8044)$. Row 3 could already see every key, so it stays at $(1.0000,\ 1.0000)$.

**The same input without the scaling:**

- Row 1's weights sharpen to $(0.6652,\ 0.0900,\ 0.2447)$ and the output moves to $(1.1547,\ 0.5795)$. The difference is this small only because $d_k$ is 2; at the $d_k = 64$ of §3.2 the weights collapse onto a single cell.

## 5 Common mistakes

1. **Leaving out the scaling.** The shapes are right and nothing raises an error. It shows up only as a loss that falls slowly or not at all. The hidden tests check several values of $d_k$.
2. **Splitting heads with a single reshape.** Right shape, wrong contents. Split as in §3.4, then move the axis.
3. **Applying the mask after the softmax.** The remaining weights no longer sum to 1. Set the scores to $-\infty$ before the softmax.
4. **Adding the residual after the norm.** The spec adds first and normalizes second. The hidden tests compare against PyTorch's own implementation.
5. **Not reseeding for each model.** If the three configurations start from different initializations there is no way to know what produced the difference. Reseed immediately before building each model.
6. **Debugging a chance-level result before asking what the model can see.** First check whether the information the label depends on reaches the model at all. If it does not, no amount of training fixes it.

## 6 How this maps to the lab tasks

**Table 4. Tasks matched to concept sections.** Which section each task draws on, and what grading looks at.

| Lab task | Concept | What grading looks at |
|---|---|---|
| Task 1. One attention | §3.1–3.3, §4 | the scaling, masked weights of exactly 0 |
| Task 2. Multi-head | §3.4 | splitting the feature axis, merge inverts split |
| Task 3. Positions and block | §3.5, §3.6 | sine and cosine layout, residual and norm order |
| Task 4. Controlled ablation | §3.5, §3.6 | one factor varies, seeding, determinism |
| Report figures 1–3 | §3.3, §3.5 | the mask's effect, an honest comparison |

## 7 Self-check

1. What does each row of the attention weight matrix sum to, and why? What can you say about the column sums?
2. You raised $d_k$ from 4 to 256 and forgot the scaling. What do the loss and the attention weights look like early in training?
3. Explain in two sentences why self-attention followed by averaging is permutation invariant. Which part of that property does adding a positional encoding break?
4. What goes wrong if you split heads with a single reshape to $(B, 4, L, 8)$, and why is no error raised?
5. A configuration lands at the majority-class rate. How do you tell a broken model from one whose input does not contain what the label depends on?

These questions are fair game for the oral check. You must be able to explain every line you submit.

## 8 Further reading

- Vaswani et al., *Attention Is All You Need*, NeurIPS 2017 `https://arxiv.org/abs/1706.03762`
- Alammar, *The Illustrated Transformer* `https://jalammar.github.io/illustrated-transformer/`
- Xiong et al., *On Layer Normalization in the Transformer Architecture*, ICML 2020 `https://arxiv.org/abs/2002.04745`